



```
In [1]: # ==== 0. INSTALL LIBRARY ====
!pip install mediapipe==0.10.7 protobuf==3.20.3 --quiet

33.6/33.6 MB 51.2 MB/s eta
0:00:00:00:0100:01
```

```
In [2]: # ==== 1. IMPORT LIBRARY ====
import os
import cv2
import numpy as np
import tensorflow as tf
import mediapipe as mp
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, roc_
from sklearn.preprocessing import label_binarize
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.applications import MobileNet
from tensorflow.keras.applications.mobilenet import preprocess_input
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, GlobalMaxPooling2D, B
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ReduceLRonPlateau
```

```
2025-05-02 20:20:40.629188: E external/local_xla/xla/stream_executor/cud
a/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to regist
er factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are writ
ten to STDERR
E0000 00:00:1746217240.892687      31 cuda_dnn.cc:8310] Unable to regist
er cuDNN factory: Attempting to register factory for plugin cuDNN when one
has already been registered
E0000 00:00:1746217240.963504      31 cuda_blas.cc:1418] Unable to regist
er cuBLAS factory: Attempting to register factory for plugin cuBLAS when
one has already been registered
```

```
In [3]: # ==== 2. SETUP PATH DAN LABEL ====
dataset_dir = "/kaggle/input/sibi-dataset/SIBI"
labels_list = sorted([d for d in os.listdir(dataset_dir) if os.path.isdir
label_map = {label: idx for idx, label in enumerate(labels_list)}
```

```
In [4]: # ==== 3. MEDIAPIPE HAND SETUP ====
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(static_image_mode=False, max_num_hands=1, min_dete

INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

```
In [5]: # ==== 4. BACKGROUND AUGMENTATION ====
def add_random_background(img):
    h, w, _ = img.shape
```

```
background = np.random.randint(0, 256, (h, w, 3), dtype=np.uint8)
alpha = np.random.uniform(0.1, 0.3)
return cv2.addWeighted(img, 1 - alpha, background, alpha, 0)
```

```
In [6]: # ==== 5. CROP TANGAN ====
def extract_hand_crop(img):
    h, w, _ = img.shape
    result = hands.process(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    if result.multi_hand_landmarks:
        x_min, y_min, x_max, y_max = w, h, 0, 0
        for lm in result.multi_hand_landmarks[0].landmark:
            x, y = int(lm.x * w), int(lm.y * h)
            x_min, y_min = min(x_min, x), min(y_min, y)
            x_max, y_max = max(x_max, x), max(y_max, y)
        pad = 40
        x_min, y_min = max(0, x_min - pad), max(0, y_min - pad)
        x_max, y_max = min(w, x_max + pad), min(h, y_max + pad)
        return img[y_min:y_max, x_min:x_max]
    return None
```

```
In [7]: # ==== 6. LOAD & PREPROCESS DATASET ====
X, y = [], []
success_count, fallback_count, total_count = 0, 0, 0

for label in tqdm(labels_list, desc="Loading data"):
    folder = os.path.join(dataset_dir, label)
    for file in os.listdir(folder):
        img_path = os.path.join(folder, file)
        img = cv2.imread(img_path)
        if img is None:
            continue
        total_count += 1
        crop = extract_hand_crop(img)
        if crop is None:
            fallback_count += 1
            h, w, _ = img.shape
            x_min, y_min = w // 4, h // 4
            x_max, y_max = 3 * w // 4, 3 * h // 4
            crop = img[y_min:y_max, x_min:x_max]

        crop = add_random_background(crop)

        gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
        blur = cv2.GaussianBlur(gray, (5, 5), 0)
        rgb = cv2.cvtColor(blur, cv2.COLOR_GRAY2RGB)
        resized = cv2.resize(rgb, (224, 224))
        processed = preprocess_input(resized) # Jangan pakai Rescaling 1

        X.append(processed)
        y.append(label_map[label])
        success_count += 1

print(f"\nTotal gambar terbaca: {total_count}")
print(f"Sukses (terdeteksi atau fallback): {success_count}")
```

```
print(f"Jumlah fallback: {fallback_count}")
```

```
Loading data: 100%|██████████| 24/24 [05:48<00:00, 14.51s/it]
```

```
Total gambar terbaca: 5280
```

```
Sukses (terdeteksi atau fallback): 5280
```

```
Jumlah fallback: 500
```

```
In [8]: # ===== 7. TRAIN-VAL SPLIT =====
```

```
X = np.array(X)
```

```
y = np.array(y)
```

```
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, ra
```

```
y_train = to_categorical(y_train, num_classes=len(labels_list))
```

```
y_val = to_categorical(y_val, num_classes=len(labels_list))
```

```
In [9]: # ===== 8. AUGMENTASI =====
```

```
datagen = ImageDataGenerator(
```

```
    rotation_range=20,
```

```
    width_shift_range=0.15,
```

```
    height_shift_range=0.15,
```

```
    zoom_range=0.2,
```

```
    brightness_range=[0.5, 1.5],
```

```
    shear_range=0.2,
```

```
    horizontal_flip=True,
```

```
    fill_mode='nearest'
```

```
)
```

```
datagen.fit(X_train)
```

```
In [10]: # ===== 9. MODEL MOBILENET =====
```

```
base_model = MobileNet(include_top=False, weights='imagenet', input_shape
```

```
for layer in base_model.layers[:-80]: # Freeze lebih sedikit
```

```
    layer.trainable = False
```

```
model = Sequential([
```

```
    base_model,
```

```
    GlobalMaxPooling2D(),
```

```
    BatchNormalization(),
```

```
    Dense(1024, activation='relu'),
```

```
    Dropout(0.3),
```

```
    BatchNormalization(),
```

```
    Dense(512, activation='relu'),
```

```
    Dense(len(labels_list), activation='softmax')
```

```
])
```

```
model.compile(optimizer=Adam(learning_rate=1e-4),
```

```
              loss='categorical_crossentropy',
```

```
              metrics=['accuracy'])
```

```
I0000 00:00:1746217644.461567      31 gpu_device.cc:2022] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 13942 MB memory:  -> de
vice: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability:
7.5
I0000 00:00:1746217644.462181      31 gpu_device.cc:2022] Created device
/job:localhost/replica:0/task:0/device:GPU:1 with 13942 MB memory:  -> de
vice: 1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability:
7.5
Downloading data from https://storage.googleapis.com/tensorflow/keras-app
lications/mobilenet/mobilenet_1_0_224_tf_no_top.h5
17225924/17225924 ————— 0s 0us/step
```

```
In [11... # ===== 10. CALLBACKS =====
callbacks = [
    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=3, verbose=0)
]
```

```
In [12... # ===== 11. TRAINING =====
history = model.fit(datagen.flow(X_train, y_train, batch_size=32),
                    validation_data=(X_val, y_val),
                    epochs=50,
                    callbacks=callbacks)
```

Epoch 1/50

```
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/
py_dataset_adapter.py:122: UserWarning: Your `PyDataset` class should call
`super().__init__(**kwargs)` in its constructor. `**kwargs` can include
`workers`, `use_multiprocessing`, `max_queue_size`. Do not pass these argu
ments to `fit()`, as they will be ignored.
```

```
self._warn_if_super_not_called()
WARNING: All log messages before absl::InitializeLog() is called are writ
ten to STDERR
```

```
I0000 00:00:1746217681.921374      126 service.cc:148] XLA service 0x7c265
801d0f0 initialized for platform CUDA (this does not guarantee that XLA w
ill be used). Devices:
```

```
I0000 00:00:1746217681.962442      126 service.cc:156]   StreamExecutor de
vice (0): Tesla T4, Compute Capability 7.5
```

```
I0000 00:00:1746217681.962470      126 service.cc:156]   StreamExecutor de
vice (1): Tesla T4, Compute Capability 7.5
```

```
I0000 00:00:1746217683.777766      126 cuda_dnn.cc:529] Loaded cuDNN versi
on 90300
```

```
1/132 ————— 1:16:24 35s/step - accuracy: 0.0312 - loss:
3.7277
```

```
I0000 00:00:1746217695.188946      126 device_compiler.h:188] Compiled clu
ster using XLA! This line is logged at most once for the lifetime of the
process.
```

132/132  **86s** 390ms/step - accuracy: 0.1218 - loss: 3.2086 - val_accuracy: 0.3892 - val_loss: 2.0089 - learning_rate: 1.0000 e-04
Epoch 2/50
132/132  **51s** 367ms/step - accuracy: 0.5250 - loss: 1.6315 - val_accuracy: 0.7898 - val_loss: 0.8251 - learning_rate: 1.0000 e-04
Epoch 3/50
132/132  **51s** 363ms/step - accuracy: 0.7436 - loss: 0.8977 - val_accuracy: 0.8807 - val_loss: 0.4194 - learning_rate: 1.0000 e-04
Epoch 4/50
132/132  **51s** 364ms/step - accuracy: 0.8226 - loss: 0.5884 - val_accuracy: 0.9195 - val_loss: 0.2780 - learning_rate: 1.0000 e-04
Epoch 5/50
132/132  **51s** 362ms/step - accuracy: 0.8626 - loss: 0.4509 - val_accuracy: 0.9214 - val_loss: 0.2436 - learning_rate: 1.0000 e-04
Epoch 6/50
132/132  **50s** 360ms/step - accuracy: 0.8975 - loss: 0.3361 - val_accuracy: 0.9366 - val_loss: 0.1876 - learning_rate: 1.0000 e-04
Epoch 7/50
132/132  **50s** 360ms/step - accuracy: 0.9069 - loss: 0.2931 - val_accuracy: 0.9508 - val_loss: 0.1607 - learning_rate: 1.0000 e-04
Epoch 8/50
132/132  **50s** 359ms/step - accuracy: 0.9297 - loss: 0.2290 - val_accuracy: 0.9574 - val_loss: 0.1367 - learning_rate: 1.0000 e-04
Epoch 9/50
132/132  **50s** 357ms/step - accuracy: 0.9401 - loss: 0.2153 - val_accuracy: 0.9612 - val_loss: 0.1231 - learning_rate: 1.0000 e-04
Epoch 10/50
132/132  **50s** 362ms/step - accuracy: 0.9426 - loss: 0.1852 - val_accuracy: 0.9631 - val_loss: 0.1142 - learning_rate: 1.0000 e-04
Epoch 11/50
132/132  **51s** 365ms/step - accuracy: 0.9481 - loss: 0.1679 - val_accuracy: 0.9735 - val_loss: 0.0907 - learning_rate: 1.0000 e-04
Epoch 12/50
132/132  **50s** 360ms/step - accuracy: 0.9611 - loss: 0.1372 - val_accuracy: 0.9697 - val_loss: 0.0991 - learning_rate: 1.0000 e-04
Epoch 13/50
132/132  **50s** 360ms/step - accuracy: 0.9580 - loss: 0.1389 - val_accuracy: 0.9706 - val_loss: 0.0937 - learning_rate: 1.0000 e-04
Epoch 14/50
132/132  **0s** 356ms/step - accuracy: 0.9635 - loss: 0.1138
Epoch 14: ReduceLR0nPlateau reducing learning rate to 1.9999999494757503

e-05.
132/132  **50s** 362ms/step - accuracy: 0.9635 - loss: 0.1139 - val_accuracy: 0.9688 - val_loss: 0.0939 - learning_rate: 1.0000 e-04
Epoch 15/50
132/132  **51s** 366ms/step - accuracy: 0.9633 - loss: 0.1202 - val_accuracy: 0.9773 - val_loss: 0.0786 - learning_rate: 2.0000 e-05
Epoch 16/50
132/132  **51s** 363ms/step - accuracy: 0.9701 - loss: 0.1123 - val_accuracy: 0.9735 - val_loss: 0.0819 - learning_rate: 2.0000 e-05
Epoch 17/50
132/132  **50s** 357ms/step - accuracy: 0.9692 - loss: 0.0968 - val_accuracy: 0.9754 - val_loss: 0.0755 - learning_rate: 2.0000 e-05
Epoch 18/50
132/132  **50s** 358ms/step - accuracy: 0.9809 - loss: 0.0698 - val_accuracy: 0.9773 - val_loss: 0.0747 - learning_rate: 2.0000 e-05
Epoch 19/50
132/132  **50s** 355ms/step - accuracy: 0.9743 - loss: 0.0753 - val_accuracy: 0.9773 - val_loss: 0.0767 - learning_rate: 2.0000 e-05
Epoch 20/50
132/132  **50s** 356ms/step - accuracy: 0.9715 - loss: 0.0910 - val_accuracy: 0.9773 - val_loss: 0.0739 - learning_rate: 2.0000 e-05
Epoch 21/50
132/132  **50s** 358ms/step - accuracy: 0.9731 - loss: 0.0818 - val_accuracy: 0.9782 - val_loss: 0.0724 - learning_rate: 2.0000 e-05
Epoch 22/50
132/132  **51s** 361ms/step - accuracy: 0.9789 - loss: 0.0706 - val_accuracy: 0.9782 - val_loss: 0.0701 - learning_rate: 2.0000 e-05
Epoch 23/50
132/132  **50s** 359ms/step - accuracy: 0.9837 - loss: 0.0676 - val_accuracy: 0.9763 - val_loss: 0.0712 - learning_rate: 2.0000 e-05
Epoch 24/50
132/132  **49s** 353ms/step - accuracy: 0.9789 - loss: 0.0677 - val_accuracy: 0.9782 - val_loss: 0.0731 - learning_rate: 2.0000 e-05
Epoch 25/50
132/132  **49s** 354ms/step - accuracy: 0.9705 - loss: 0.0811 - val_accuracy: 0.9773 - val_loss: 0.0687 - learning_rate: 2.0000 e-05
Epoch 26/50
132/132  **50s** 357ms/step - accuracy: 0.9860 - loss: 0.0510 - val_accuracy: 0.9763 - val_loss: 0.0711 - learning_rate: 2.0000 e-05
Epoch 27/50
132/132  **49s** 355ms/step - accuracy: 0.9798 - loss: 0.0633 - val_accuracy: 0.9763 - val_loss: 0.0753 - learning_rate: 2.0000

e-05
Epoch 28/50
132/132  **50s** 356ms/step - accuracy: 0.9849 - loss: 0.0524 - val_accuracy: 0.9763 - val_loss: 0.0683 - learning_rate: 2.0000e-05
Epoch 29/50
132/132  **49s** 354ms/step - accuracy: 0.9861 - loss: 0.0494 - val_accuracy: 0.9773 - val_loss: 0.0674 - learning_rate: 2.0000e-05
Epoch 30/50
132/132  **49s** 353ms/step - accuracy: 0.9809 - loss: 0.0619 - val_accuracy: 0.9792 - val_loss: 0.0682 - learning_rate: 2.0000e-05
Epoch 31/50
132/132  **49s** 355ms/step - accuracy: 0.9869 - loss: 0.0429 - val_accuracy: 0.9782 - val_loss: 0.0657 - learning_rate: 2.0000e-05
Epoch 32/50
132/132  **50s** 355ms/step - accuracy: 0.9838 - loss: 0.0584 - val_accuracy: 0.9792 - val_loss: 0.0628 - learning_rate: 2.0000e-05
Epoch 33/50
132/132  **50s** 356ms/step - accuracy: 0.9865 - loss: 0.0406 - val_accuracy: 0.9801 - val_loss: 0.0639 - learning_rate: 2.0000e-05
Epoch 34/50
132/132  **50s** 357ms/step - accuracy: 0.9877 - loss: 0.0436 - val_accuracy: 0.9801 - val_loss: 0.0672 - learning_rate: 2.0000e-05
Epoch 35/50
132/132  **0s** 358ms/step - accuracy: 0.9805 - loss: 0.0519
Epoch 35: ReduceLR0nPlateau reducing learning rate to 3.999999898951501e-06.
132/132  **51s** 364ms/step - accuracy: 0.9805 - loss: 0.0519 - val_accuracy: 0.9801 - val_loss: 0.0663 - learning_rate: 2.0000e-05
Epoch 36/50
132/132  **50s** 357ms/step - accuracy: 0.9924 - loss: 0.0330 - val_accuracy: 0.9801 - val_loss: 0.0636 - learning_rate: 4.0000e-06
Epoch 37/50
132/132  **50s** 360ms/step - accuracy: 0.9881 - loss: 0.0410 - val_accuracy: 0.9792 - val_loss: 0.0621 - learning_rate: 4.0000e-06
Epoch 38/50
132/132  **49s** 351ms/step - accuracy: 0.9866 - loss: 0.0396 - val_accuracy: 0.9811 - val_loss: 0.0610 - learning_rate: 4.0000e-06
Epoch 39/50
132/132  **50s** 357ms/step - accuracy: 0.9893 - loss: 0.0410 - val_accuracy: 0.9801 - val_loss: 0.0609 - learning_rate: 4.0000e-06
Epoch 40/50
132/132  **49s** 354ms/step - accuracy: 0.9896 - loss:

```

0.0319 - val_accuracy: 0.9811 - val_loss: 0.0597 - learning_rate: 4.0000
e-06
Epoch 41/50
132/132  50s 360ms/step - accuracy: 0.9861 - loss:
0.0399 - val_accuracy: 0.9801 - val_loss: 0.0594 - learning_rate: 4.0000
e-06
Epoch 42/50
132/132  50s 359ms/step - accuracy: 0.9886 - loss:
0.0403 - val_accuracy: 0.9792 - val_loss: 0.0600 - learning_rate: 4.0000
e-06
Epoch 43/50
132/132  52s 371ms/step - accuracy: 0.9904 - loss:
0.0321 - val_accuracy: 0.9811 - val_loss: 0.0585 - learning_rate: 4.0000
e-06
Epoch 44/50
132/132  50s 356ms/step - accuracy: 0.9850 - loss:
0.0490 - val_accuracy: 0.9820 - val_loss: 0.0591 - learning_rate: 4.0000
e-06
Epoch 45/50
132/132  49s 353ms/step - accuracy: 0.9914 - loss:
0.0339 - val_accuracy: 0.9811 - val_loss: 0.0595 - learning_rate: 4.0000
e-06
Epoch 46/50
132/132  0s 349ms/step - accuracy: 0.9873 - loss: 0.0
490
Epoch 46: ReduceLR0nPlateau reducing learning rate to 7.999999979801942
e-07.
132/132  50s 356ms/step - accuracy: 0.9873 - loss:
0.0489 - val_accuracy: 0.9792 - val_loss: 0.0604 - learning_rate: 4.0000
e-06
Epoch 47/50
132/132  50s 359ms/step - accuracy: 0.9890 - loss:
0.0387 - val_accuracy: 0.9792 - val_loss: 0.0603 - learning_rate: 8.0000
e-07
Epoch 48/50
132/132  50s 358ms/step - accuracy: 0.9910 - loss:
0.0332 - val_accuracy: 0.9792 - val_loss: 0.0599 - learning_rate: 8.0000
e-07

```

```

In [13... # ==== 12. SIMPAN MODEL ====
model.save("/kaggle/working/sibi_final2.h5")
np.save("/kaggle/working/labels_list.npy", labels_list)

```

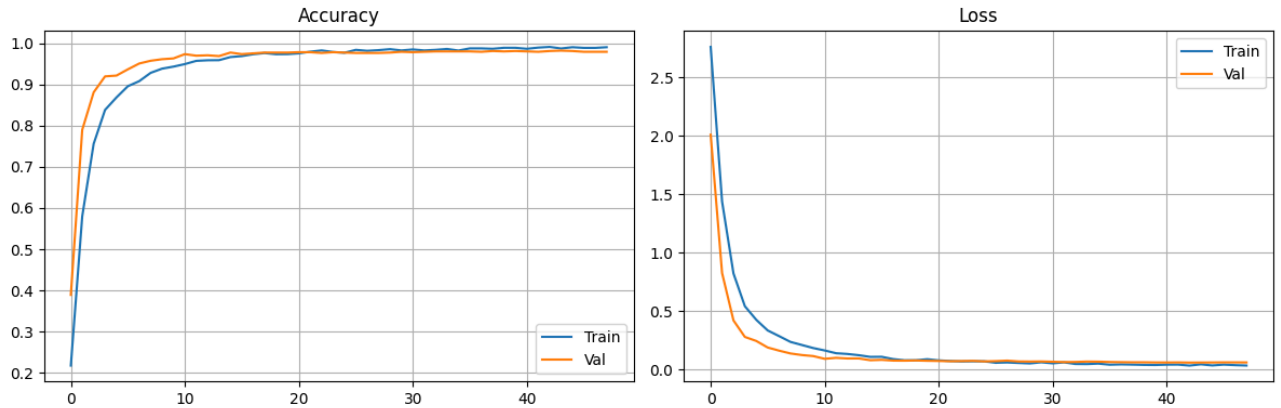
```

In [14... # ==== 13. VISUALISASI TRAINING ====
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title('Accuracy')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train')

```

```
plt.plot(history.history['val_loss'], label='Val')
plt.title('Loss')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



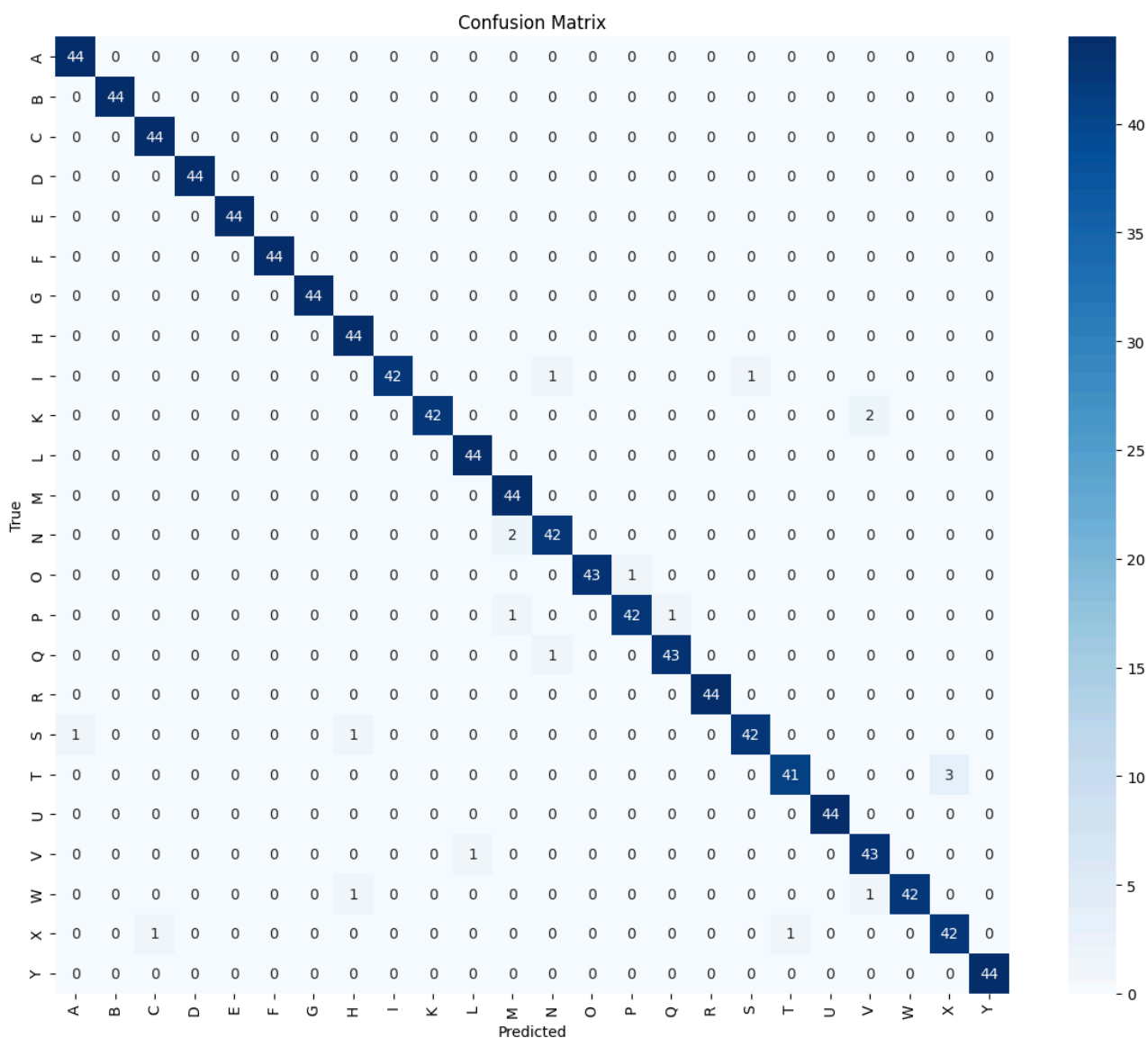
```
In [15... # ===== 14. EVALUASI =====
y_pred = model.predict(X_val)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_val, axis=1)

print("Classification Report:\n")
print(classification_report(y_true, y_pred_classes, target_names=labels_l

conf_matrix = confusion_matrix(y_true, y_pred_classes)
plt.figure(figsize=(12, 10))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=labels_list, yticklabels=labels_list)
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

Classification Report:

	precision	recall	f1-score	support
A	0.98	1.00	0.99	44
B	1.00	1.00	1.00	44
C	0.98	1.00	0.99	44
D	1.00	1.00	1.00	44
E	1.00	1.00	1.00	44
F	1.00	1.00	1.00	44
G	1.00	1.00	1.00	44
H	0.96	1.00	0.98	44
I	1.00	0.95	0.98	44
K	1.00	0.95	0.98	44
L	0.98	1.00	0.99	44
M	0.94	1.00	0.97	44
N	0.95	0.95	0.95	44
O	1.00	0.98	0.99	44
P	0.98	0.95	0.97	44
Q	0.98	0.98	0.98	44
R	1.00	1.00	1.00	44
S	0.98	0.95	0.97	44
T	0.98	0.93	0.95	44
U	1.00	1.00	1.00	44
V	0.93	0.98	0.96	44
W	1.00	0.95	0.98	44
X	0.93	0.95	0.94	44
Y	1.00	1.00	1.00	44
accuracy			0.98	1056
macro avg	0.98	0.98	0.98	1056
weighted avg	0.98	0.98	0.98	1056



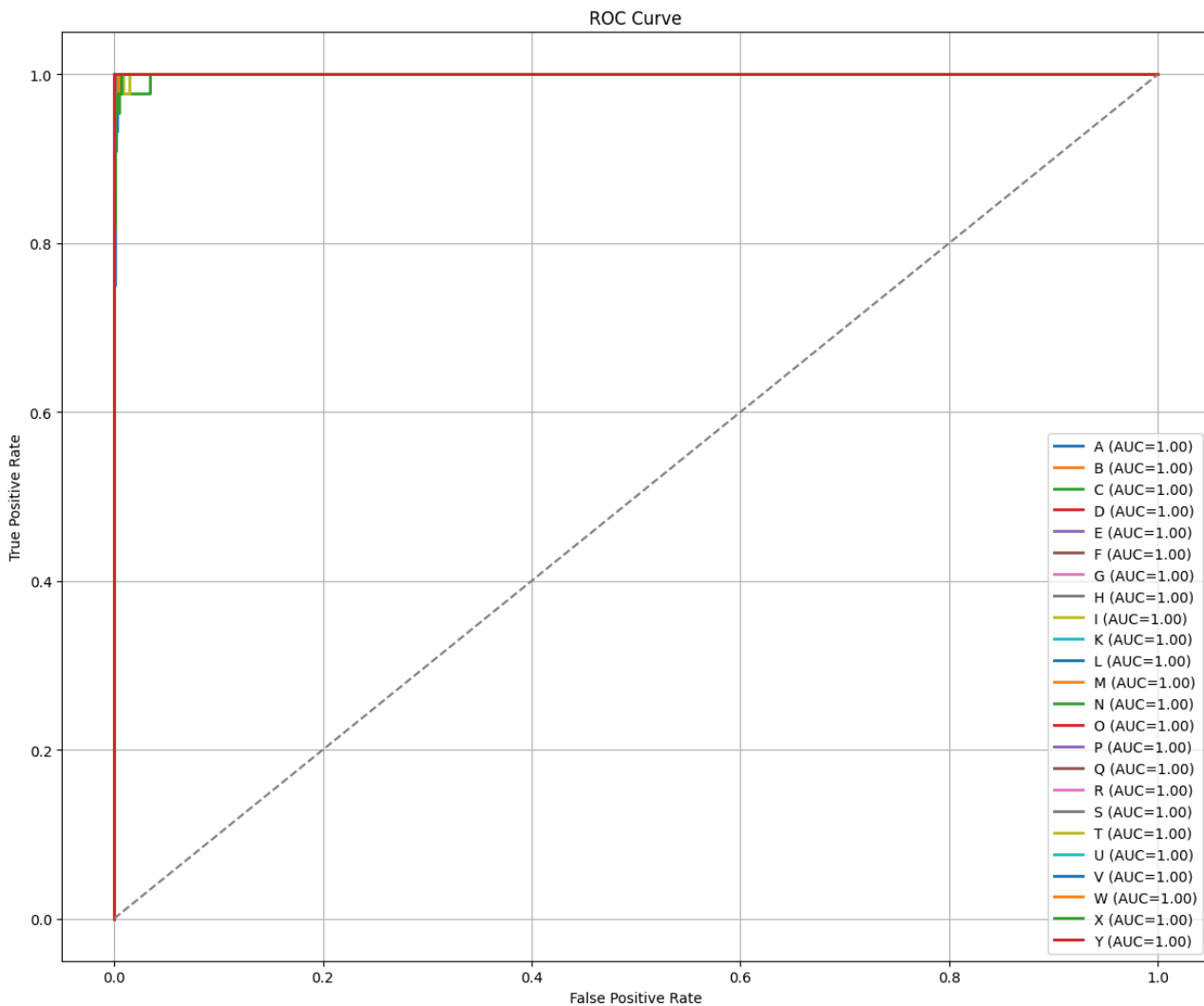
```
In [16... # ===== 15. ROC CURVE & AUC =====
y_true_bin = label_binarize(y_true, classes=np.arange(len(labels_list)))
fpr, tpr, roc_auc = {}, {}, {}

plt.figure(figsize=(12, 10))
for i in range(len(labels_list)):
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_pred[:, i])
    roc_auc[i] = roc_auc_score(y_true_bin[:, i], y_pred[:, i])
    plt.plot(fpr[i], tpr[i], lw=2, label=f'{labels_list[i]} (AUC={roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], color='gray', linestyle='--')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
```

```
plt.show()

print(f"Average Macro AUC: {np.mean(list(roc_auc.values())):.4f}")
```



Average Macro AUC: 0.9999